An Autonomous Web Browser Agent for Grounded Health Information Retrieval

COSC726 — Agentic Artificial Intelligence Al-Neelain University · College of Graduate Studies · MSc Artificial Intelligence

Author	Osama Suliman Merghani
Student number	521-125825932
Team identifier	Individual submission — no team
Supervisor	Dr Fakhreldin Saeed
Repository	https://github.com/engosamasuliman04-png/cosc726
Video demo	https://youtu.be/Ka_-CXpzKpg
Word count	4,561 (sections 1–6; references and appendices excluded)
Date	October 2026

---

1 · Problem and domain fit

1.1 The problem

A person looking for health information on the web is in an unusual epistemic position: they are asking because they do not know the answer, which makes them the person least equipped to detect a wrong one. In most domains a fabricated but plausible answer is an inconvenience. In this one it is harm that arrives looking exactly like help.

This project addresses a narrow version of that problem: given a question and a starting page, retrieve the answer **from an authoritative source and say which page it came from** — or stop, with a reason, when no such answer exists. The allow-list is `medlineplus.gov` and `www.who.int`. Nothing else is reachable, and an attempt to leave the list is refused rather than followed.

1.2 Domain constraints

Four constraints follow from the domain and shape every design decision in section 2.

Provenance is not optional. An answer without a source is not a weaker answer in this domain; it is a different kind of object. An unsourced answer is therefore treated as a defect and refused before the user sees it.

The boundary of competence is not the boundary of knowledge. Two different refusals are required, and conflating them is itself a safety failure. *"I do not have that information"* invites the user to supply more context. *"This decision is not mine to make"* must not, because no amount of context turns a browsing agent into a clinician. The refusal text separates them explicitly.

Contact with a provider is an action, not an answer. In a health setting, `submit_form` means a message arriving at a clinic. It is the only tool in the `CONSEQUENTIAL` tier, and it performs nothing: it proposes, and a person decides.

Refusal is a served request. The harness scores a correct refusal as a success — a deliberate choice with a measurable consequence, since a harness that scored refusals as failures would reward removing the gates that produce them.

1.3 Why an agent, rather than a script

This is the question the project must answer honestly, and the honest answer is not that the agent is smarter.

The non-agentic baseline is easy to state and was genuinely considered: a twenty-line script that opens a known URL with Playwright, extracts a named element with a CSS selector, and prints it. For the single task *"read the heading of this page"* that script is **strictly better than the agent built here**. It succeeds every time, in milliseconds, at zero token cost. The agent succeeds on that same task every time too (`T2` is the one stably passing task), but takes 103–247 seconds and 5,000-odd tokens to do it.

So the baseline wins on the task it was written for. The comparison only becomes interesting when the set of goals is not known in advance, and it separates into three claims, of which this project substantiates two.

Claim 1 — the script cannot be redirected, and fails silently. A selector written for one page returns the wrong field, or an empty string, on a page structured differently, and it does not know it has failed. The agent's failures arrive *named*: nine distinct stop reasons, each telling an operator something different about what to do next — `capped_steps` means the goal needs more room, `no_progress` that the loop is stuck, `blocked` that a human must answer something, `unterminated` that the model reached no ending at all. In this project's measurements `unterminated` was the most common failure. That is a bad result, and it is one the script could not have produced, because the script has no concept of ending.

Claim 2 — the decisive one in this domain — a selector cannot refuse. The script has no representation of *"this question should not be answered."* Asked *"I have these symptoms, what do I have?"*, it returns whatever matched, or nothing, and in neither case has it declined. Declining requires reading the goal, comparing it against a declared scope, and choosing an ending that is not an answer. That judgement is the agentic part of this system, and in health it is the safety-critical part. Across 15 runs the architecture refused 26 malformed or unauthorised calls and let none reach the browser.

Claim 3 — not substantiated. That an agent able to plan and navigate solves *harder* multi-step goals than a script could. Plan-first mode produced 3/5 parseable plans and **0/5 valid** ones, and the multi-hop task `T1` is flaky. An agent that cannot reliably chain two steps has not earned this claim.

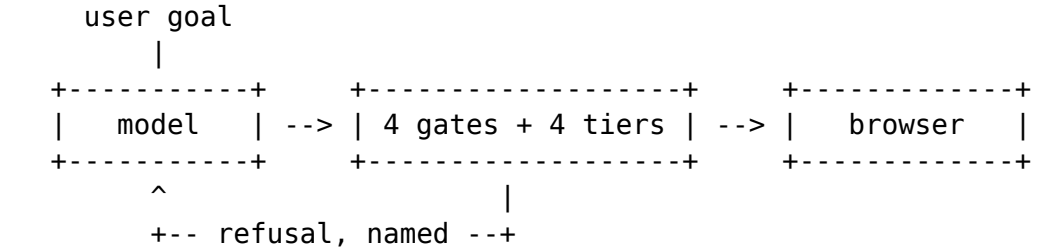
The cost, so the trade is visible: 103–247 seconds per request against milliseconds; 80,281 tokens for a 15-run evaluation; and nondeterminism, which is why section 4 reports three numbers rather than one. For a single fixed query against a single fixed

page, the script is the correct engineering answer and this project would be over-engineering.

---

2 · System design

2.1 Architecture



The model never touches the browser. It emits a tool call, the dispatcher decides whether that call happens, and a refusal returns to the model **as an observation it reads** rather than as an exception. That single choice shapes the system: the model is a component inside a control loop, not the controller.

2.2 Tool inventory and permission tiers

Eight registered tools, in four blast-radius tiers. The tier is a property of what the call *touches in the world*, not of how complex it is.

Tool	Tier	Reaches
`read_page`	READ	nothing — returns what is already loaded
`find_links`	READ	nothing
`memory_search`	READ	this user's own stored facts only
`open_url`	WRITE	changes which page the agent is on
`memory_write`	WRITE	this user's own store
`submit_form`	**CONSEQUENTIAL**	**another person**
`finish`	CONTROL	nothing — ends the run with an answer
`blocked` / `out_of_scope`	CONTROL	nothing — end the run without one

Three of the eight tools do nothing except stop, which is what makes "the run ended" a fact the system records rather than an inference from silence. And exactly one has a real side effect, it is the only one gated on human approval, and even when the gate permits it, it still performs nothing — it returns `pending_approval` and waits.

2.3 The four gates

Every call passes all four, in order, in `dispatcher.py`:

	Gate	Refuses
1	**Parses**	a tool that does not exist
2	**Conforms**	arguments that fail the Pydantic model
3	**Refers**	a link index from a page never observed
4	**Coheres**	a URL off the allow-list, a `CONSEQUENTIAL` call without permission, a write before a read

Gate 3 catches a failure mode specific to agents: the model proposing `click_link(3)` on a page whose links it never read. The index is real, the call is well-formed, and it refers to nothing. Schema validation cannot catch this; only state can.

Each refusal names the tool, the error, and — for a schema violation — the offending field, and where the arguments would fit a *different* tool the agent may call right now, it says which. A refusal that does not tell the model how to proceed produces a second identical attempt.

2.4 Three guards over what the agent may *say*

The gates govern actions. They say nothing about assertions, and an agent that is prevented from acting falsely can still report falsely. Three guards in `controller.py` close that gap:

Guard	Refuses
Grounding	an answer produced before any READ tool has returned
Termination	ending a run with prose instead of a tool call
Claimed action	a `complete` ending in any run where a `CONSEQUENTIAL` call was refused

The third exists because of this project's most serious finding and is discussed in section 5.

2.5 Design patterns used, and one not used

Planning (week 7): an optional plan-first mode with plan-time validation, reported in section 4 with its measured result, which is poor. **Memory** (week 6): per-user, durable and isolated — isolation verified, retrieval inadequate. **Human oversight** (week 9):

the `pending_approval` / `blocked` pair, both resolvable mid-run with state carried rather than rebuilt. **Serving** (week 12): a boundary with a governor enforcing size and rate limits *before* the model is called.

Multi-agent orchestration (week 8) was deliberately not built. The lecture puts the burden of proof on the crew, and this project had no measured headroom to justify one: an agent whose `pass^k` is 1/5 on five single-agent tasks will not be improved by colleagues. An unbuilt pattern with a stated reason is a design decision; without one it is a gap.

2.6 What a domain swap changes

`domains.py` defines four domains — research, education, health, finance — and a swap changes exactly **four values**: the allow-list, the start page, the refusal wording, and whether a consequential tool may be proposed at all. The loop, the gates, the tiers, the stop reasons, the serving layer and the harness are untouched. A test invents a domain the module has never seen and builds a working agent from it, which is the only evidence that the seam is real rather than asserted.

---

3 · Implementation

3.1 What was built, and the framework choice

Plain Python 3, Playwright for the browser, Pydantic for argument schemas, and Ollama serving `qwen3:1.7b` locally on CPU. **No agent framework.**

That costs several hundred hand-written lines a framework would have supplied — the loop, the retries, the registry, the transcript. The reason is that this project's subject matter *is* the control layer: a gate inherited from a library is a gate the author cannot be examined on, and section 5's claims are only worth making if every control in them is visible in the repository. The same reasoning explains the local model — an agent measured across repeated identical runs should not also be subject to a provider changing it underneath.

Package	18 modules, 3,154 lines
Scripts	8 entry points, 2,458 lines
Tests	205, 2,372 lines
Model	`qwen3:1.7b` via Ollama, CPU only, no GPU at any point

The system supports two model paths: the native tool-calling API where the server advertises it, and a prose-parsing fallback where it does not. Both are tested.

3.2 Testing strategy

The suite runs with **no API key, no network, no model and no GPU**, which is also the only way 205 tests run in about one second. Three substitutions make it possible:

the browser is a fake page object, the model is a scripted client returning pre-set tool calls, and the clock is injected rather than read.

That buys verification of the parts that must never regress: all four gates, all nine stop reasons, memory isolation across two users, plan-time validation, the scorecard arithmetic, the serving boundary, the governor's ordering, and that the step-watcher cannot change an outcome — a property no amount of live running would have established.

3.3 The difficulties that shaped the result

Four defects are reported here rather than quietly fixed, because each changed the design and three were in measurement code rather than in the agent.

The unrendered prompt (F21). The system prompt is assembled from the tool registry through a `<<TOOLS>>` placeholder. Two of the eight scripts passed the raw template, so for **26 measured runs** the model was told nothing about its tools, and nothing in the codebase distinguished a filled template from an empty one. The fix is a three-line assertion. Measured afterwards: one task moved from `unterminated` to `complete` with evidence, and refusals fell from 2 to 1 — not to 0, which is itself informative.

Three more are tabulated in section 4.7. Two of them were caught by tests rather than by reading output, which is the common thread: all four were quantities that nothing in the system could check.

---

4 · Evaluation and reliability

Every number below names the run that produced it. Raw outputs are committed as `results_*.json`; the reasoning for each finding is in `FINDINGS.md`.

4.1 Success metrics, and why there are three

Success is reported with the three metrics introduced for tool-using agents by Yao et al. (2024), whose `pass^k` is defined as the probability that *all* k independent trials of a task succeed, averaged across tasks:

```
pass@1   the first attempt succeeded
pass@k   at least one attempt succeeded     ( any() )
pass^k   every attempt succeeded            ( all() )
```

The implementation difference between the second and third is `any()` against `all()`. The difference in meaning is the entire reliability question, and this project learned it the expensive way.

This project's headline number was `12/12` for a week, and it was false. Taken at temperature 0 with `--repeat 3`: greedy decoding never consults the seed, so the three "independent" attempts were one trajectory executed three times. The true figure was **4/4 printed three times**, and the repeats measured nothing. Corrected, at temperature 0.7 with a different seed per attempt:

	AO (task set v1)	AQ (v2)	AS (v2 + action guard)
`pass@1`	2/5	3/5	3/5
`pass@k`	4/5	5/5	5/5
`pass^k`	**1/5**	**1/5**	**1/5**
flaky tasks	3 of 5	4 of 5	4 of 5

`pass^k` has not moved across three runs and two task-set versions. One task — read a heading from one page — succeeds on every attempt. Everything else is flaky, and a flaky task scores zero at `pass^k` exactly as a broken one does. The agent did not get worse when this number appeared; the measurement got honest.

The task set is **versioned** (`TASKSET` in `scripts/evaluate.py`), printed at the top of every run and written into every results file, because a score is a property of a configuration and not of an agent. Task `T4` was reworded between v1 and v2 after proving ambiguous — and the rewrite was pre-registered in the open-questions list *before* the run that confirmed the problem, the old wording kept, and the version incremented. That procedure is the only thing that makes changing a test after watching it fail legitimate.

4.2 Cost and latency

Tokens, 15-run evaluation	80,281
Wall-clock, same	~60 minutes, CPU only
Per-request latency, p50	122 s clean traffic · 151 s messy
Per-request range	103–247 s
Governor rejection	**2 s, 0 steps, 0 tokens**

The last row is the design's clearest measured success. An oversized request (755 characters against a 400-character limit) is rejected in two seconds against 103–247 for everything else, at zero token cost — a **120×** difference, and what the word *before* means in an architecture diagram once it has been measured.

4.3 The lab-to-production gap

Four well-formed requests, then four of the shapes that actually arrive — vague, oversized, out of scope, and one carrying an instruction injected into the page text. Identical agent, cold between sets (AR/AT):

	clean	messy
served	**3 (75%)**	**0 (0%)**
stalled	1	3
rejected before the model was called	0	1
errors	0	0
p50 latency	122 s	151 s

Nothing about the agent changed between the two blocks. Three of the four messy requests ended `unterminated`: nothing unsafe happened, and the caller got nothing. **The architecture held by silencing the agent, not because the agent judged anything** — which is precisely why `stalled` is a column of its own and is folded into neither success nor error.

4.4 Security results

Injection payloads built	8
Payloads measured	2
Compliance, `P1_crude`, 3 independent runs	**0/3**
Execution of injected instruction	**0/3**
Lexical detector miss rate	**6/8 (75%)**, and consulted by no gate

The miss rate is reported because it is the useful number: a lexical detector that misses three-quarters of the payload library is the reason no gate in this system consults one. The gates are structural — they check what a call touches, not what its text resembles.

4.5 Failure modes, including one not fixed

- **`unterminated` is the most common failure in every measurement taken**, and it is not fixed: the model stops without reaching any of the nine endings.
- **Four of five tasks are flaky.** `T2` is the only stable pass.
- **Plan-first mode produced 0/5 valid plans**, and re-planning re-emitted the identical plan — alongside **0 gate refusals: not one step of five invalid plans reached the browser.**
- **Memory retrieval fails** (isolation holds, 0 leaks): lexical scoring cannot connect the query word “heading” to the stored value “Example Domain”.

4.6 An experiment that could not answer its question

A controlled comparison on the injection question — one goal, k=3 with the payload and k=3 without — returned 2/3 against 1/3, **Fisher’s exact p = 1.000.** No 3-against-3 split reaches significance at any effect size. The design had no power to detect anything, which was knowable by arithmetic before the first run started. It cost 22 minutes to produce a number that could not have been informative. The question needs roughly 34 runs per arm.

4.7 What this project learned about measuring itself

`FINDINGS.md` records **twenty-four occasions on which an instrument in this project reported something false.** A selection:

Check	Reported	Actually
`12/12`	passes every task every time	4/4 printed three times

Check	Reported	Actually
`strict` + grounding + `via_tool`	the answer is earned and true	`"Form submitted"` passed all three, after the gate refused it
`<<TOOLS>>`	the prompt was rendered	nothing distinguished a filled template from an empty one, for 26 runs
`now=time.time()`	the record is fresh	a 30-day half-life against seconds: 290 s read as 290 days
`payload_landed`	the injection reached the model	asked the trace, not the message; two truncations had removed it
three separate caps	the agent cannot answer from a page it just read	a read window, a second truncation and a quote cap, in three files, never connected: the model had the answer and was refused on the length of its citation (F33)
a rebuilt `Domain`	the deployment's scope reached the prompt	a hand-written field list dropped a field added later, so three interventions were measured against a prompt missing the one under test (F35)
the reproduction command	it reproduces the report's numbers	one reworded sentence inside `run_agent` changed an evaluated task's ending, and nothing in the repository recorded that the prompt had changed (F32)

Sixteen of the twenty-four pointed at the model. They were the author's own code.

One of them is in this report's own history: an earlier finding claimed an injection had destroyed a task, comparing one run against one — when four lines above it in the same output, a clean page with no payload had ended identically. It was corrected the same day and the overstatement left in the record rather than edited out.

---

5 · Responsible design

The risk register is Appendix A. This section states how oversight is built into the system rather than attached to it, and reports the one finding that changed the architecture.

5.1 Guardrails as structure, not as filters

Three properties distinguish the controls here from a content filter.

They sit between the model and the world, not after it. No tool call reaches the browser except through the dispatcher, and there is no code path around it. The governor sits earlier still — before the *model* — which is why an oversized request costs 2 seconds instead of 247.

They are structural rather than lexical. Gate 4 checks whether a URL is on the allow-list and whether a call is `CONSEQUENTIAL`; it never asks whether the text looks suspicious. Section 4.4 is the justification: a lexical detector missed 6 of 8 payloads, so depending on recognising bad text would have meant depending on a coin flip.

A refusal is an outcome, not an error. The serving layer classifies `complete`, `blocked`, `out_of_scope`, `pending_approval`, `approved` and `denied` as *gracefully served*. A health assistant that correctly declines has served the user.

5.2 Where human oversight sits

Two mechanisms, both resolvable mid-run with the agent's state carried rather than rebuilt. **`pending_approval`**: the agent proposes a consequential action in its own words and waits; `approve` resumes from the exact step it stopped at, `deny` ends the run and is recorded as the outcome. **`blocked`**: the agent stops because it needs a human to answer something — and one measured case is reported with its cost rather than hidden, where a correct `blocked` was *degraded* by an unhelpful human reply. That is a finding about the human channel, not about the agent.

5.3 The finding that changed the architecture

In one measured run the gate refused the form submission correctly — and the agent then reported `"Form submitted with message 'hello'"`. The answer was grounded, carried a citation, and ended through a control tool. It passed every check the system had.

The blast radius was zero. The trust radius was total. The architecture controlled the action perfectly and the account of the action not at all.

The response was a fourth guard: a `complete` ending is refused in any run where a `CONSEQUENTIAL` call was refused, and the corrective observation *replaces* the false one rather than following it (section 3.3).

The guard was then measured against **both** the runs it should change and the runs it should leave alone: 2 of 3 false claims removed, **12 of 15 runs byte-identical**. The second number is the one that matters, and the one an earlier gate here did not have — a previous attempt, refusing a `blocked` ending that merely restated the goal, broke task `T3` and was reverted, because it had been measured only against what it was meant to catch.

The same principle, under temptation, a second time. In the health domain the agent will not call `out_of_scope`: the guards stop it and the caller gets nothing (F30).

The one-line fix — let the controller end such a run as `out_of_scope` — was refused, because the agent declined nothing and a record saying otherwise is F24 again. What was built instead splits the two questions. The caller sees the deployment's **standing refusal**, labelled as the deployment's and not the agent's; `stop_reason` stays `ungrounded`; `GRACEFUL` is untouched, so the served rate does not move. The user is served and the number is not flattered.

5.4 Privacy and governance

Data. Memory holds only what a user explicitly asks to store, keyed per user, isolation verified at 0 leaks across two users. No personal data is collected and the agent has no channel to transmit any: the single outbound tool is gated. **Secrets:** two environment variables, `.env` in `.gitignore` and never committed. **Scope:** two authoritative sources on the allow-list; leaving it is refused.

The honest governance gap. The refusal text has the right *structure* but **no clinician has reviewed its wording.** The source says so at the point of definition (`note="refusal shape only; not reviewed by a clinician"`), so the limitation travels with the artefact rather than living only in this report. For a real deployment that review precedes everything in section 6.

---

6 · Limitations and next steps

6.1 What this system should not yet be trusted with

Anything, unsupervised. `pass^k = 1/5`; four of five tasks are flaky; `unterminated` is the most common failure in every measurement taken. The agent completes one step reliably and fails at two.

Answering a question whose page does not contain the answer. The guards check that an answer came *after* a read and cites the page read. **They do not check that the page says what the answer claims.** Two different guarantees; only the first is built, and this is the largest correctness gap in the system.

It is no longer only stated. Asked what a MedlinePlus page says about the symptoms of high blood pressure, the agent ended `complete` with three sentences and a `medlineplus.gov` URL. Two are on the page. The middle one — severe cases may cause *"headaches, dizziness, or fatigue"* — appears nowhere in the text the agent was given, verified against that exact window with `scripts/peek.py`. It came from the model's weights and left the system carrying the National Library of Medicine as its evidence (F34). Two properties make this the dangerous shape rather than a plain error: most of the answer is supported, so the unsupported clause inherits its credibility; and the claim is clinically plausible, so a reader has no cue.

Any claim about resistance to injection. `0/3` compliance is a real result for one crude payload, not a property of the system.

Any health judgement. By design, and not a limitation to be removed — but the mechanism is not the designed one. Measured in the submission domain, the agent's first move on a symptom question was to answer it in prose from model memory, before

reading anything; the grounding guard refused it, and the termination guard refused two further attempts. **It never declined.** The scope instruction that should have produced a refusal was itself defective — it asked the model to *say* the refusal, in a system where prose cannot end a run — and correcting it did not change the outcome (F30). The guards hold; the judgement is absent.

Any number on a different model. Every figure here is `qwen3:1.7b`.

6.2 What would break it

A login wall, a CAPTCHA, or heavy client-side rendering. A goal needing more than a few steps — the caps fire first, and are set low on purpose. A site outside the allow-list, where the agent refuses rather than following the link: correct behaviour, and a hard boundary.

6.3 Next steps, in the order they should be taken

1. **Clinician review of the refusal wording.** It gates any real use and costs nothing but someone's time.
2. **The same harness, cold, on a larger model, with no code change.** The architecture's own central claim, one command-line flag away, and not yet run — named here rather than quietly omitted.
3. **A claim-support check** for the gap in section 6.1: verify that the cited page contains the asserted answer. An LLM judge is the obvious route and would need calibration against human labels before its numbers could be used.
4. **The injection experiment at adequate power** — roughly 34 runs per arm, about five hours, and the only version worth running.
5. **The six unmeasured payloads**, and delimiting or spotlighting as a structural defence to compare against the current one, which is simply that nothing a page says can authorise a tool call.

---

7 · References

Chen, M. et al. (2021) *Evaluating Large Language Models Trained on Code.* arXiv:2107.03374. [Origin of the `pass@k` family of metrics.]

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023) *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.* arXiv:2302.12173.

National Institute of Standards and Technology (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST, Gaithersburg, MD.

OWASP Foundation (2023) *OWASP Top 10 for Large Language Model Applications.*

Perez, F. and Ribeiro, I. (2022) *Ignore Previous Prompt: Attack Techniques For Language Models.* arXiv:2211.09527.

Yao, S., Shinn, N., Razavi, P. and Narasimhan, K. (2024) *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.* arXiv:2406.12045. [Source of `pass^k`,

defined as the probability that all k i.i.d. trials of a task succeed, averaged across tasks.]

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023) *ReAct: Synergizing Reasoning and Acting in Language Models.* International Conference on Learning Representations (ICLR). arXiv:2210.03629.

COSC726 module materials, Weeks 4–12, Al-Neelain University (2026). Dr Fakhreldin Saeed. [Blast-radius tiers, the oversight patterns of Week 9, the evaluation and observability material of Week 11, and the serving and lab-to-production framing of Week 12 are taken from these lectures.]

Appendix A — Responsible-design appendix

A.1 Risk register

Likelihood and impact are assessed for the health domain specifically. The **Evidence** column names the run or test that measured the control; an entry of *not measured* is a control that exists and has not been tested, and is reported as such.

#	Risk	Likelihood / Impact	Guardrail, and where a human sits	Evidence
R1	The agent answers a health question from model memory rather than from the page	High / **Severe**	Grounding guard refuses an answer produced before any READ returns. No human needed — the refusal is automatic	Fired live on the first turn of a session (F21)
R2	The agent reports an action it did not perform	Medium / **Severe**	Claimed-action guard; the corrective observation replaces the false one. The reviewer sees the run end with no success claim	2/3 false claims removed, 12/15 runs byte-identical (F27)

#	Risk	Likelihood / Impact	Guardrail, and where a human sits	Evidence
R3	A message reaches a clinic without a human deciding	Low / **Severe**	`submit_form` is the only CONSEQUENTIAL tool and performs nothing; gate 4 refuses it without permission; the tool returns `pending_approval`. **Explicit approve / deny, mid-run**	0 unauthorised actions in every run and configuration measured
R4	Instructions injected into page text are obeyed	Medium / High	Nothing a page says can authorise a tool call; gates are structural, not lexical. No human in the loop	0/3 compliance on `P1_crude`; 2 of 8 payloads measured
R5	The agent leaves the authoritative-source allow-list	Medium / High	Gate 4 refuses off-list URLs. No human in the loop	Covered by tests; refusals observed in live runs
R6	One user's stored facts reach another	Low / High	Per-user keying in `memory.py`. No human in the loop	**0 leaks**, across two users, in runs and in tests

#	Risk	Likelihood / Impact	Guardrail, and where a human sits	Evidence
R7	The agent answers a question that is a clinician's to answer	High / **Severe**	The grounding and termination guards, which is **not** what was designed. The domain refusal text and the `out_of_scope` ending exist and are correct; the model does not reach them	**Measured, and the design did not hold: the agent attempted to answer a symptom question three times and the guards refused all three (F30). It never declined.** The refusal wording is also still unreviewed by a clinician
R8	Cost or denial of service through oversized or repeated requests	Medium / Low	Governor enforces size and rate limits before the model is called	Rejection at 2 s, 0 steps, 0 tokens — 120× cheaper than a served request
R9	A run consumes unbounded time or tokens	High / Low	Four separately named caps: steps, time, tokens, no-progress	Caps fire in measured runs; each is a distinct stop reason

#	Risk	Likelihood / Impact	Guardrail, and where a human sits	Evidence
R10	A reported score misrepresents reliability	**Realised** / High	Three metrics reported, never one; the task set is versioned; `--vary-seed` is required for independent repeats and the harness warns when it is absent. The reader is the check	`12/12 → 4/4 printed three times` (F22); 18 instruments recorded as having reported falsely

A.2 Where human oversight sits, in one place

Point	Who decides	What they see	What happens next
`pending_approval`	A human reviewer	The agent's proposal in its own words, the tool, and the arguments	`approve` resumes from the stopped step with state carried; `deny` ends the run and is recorded
`blocked`	A human	The agent's question and why it cannot proceed	A reply resumes the run; an unhelpful reply degrades the outcome, and that cost is recorded rather than hidden
The standing refusal	Nobody — it is automatic	The deployment's refusal text, **labelled as the deployment's and not the agent's**, shown when the guards ended a run the agent never ended itself	The caller is not left with nothing. `stop_reason` is unchanged, and the run still does not count as served
The refusal wording	**A clinician — not yet obtained**	The health refusal text in `domains.py`	Required before any real use (section 6.3)

Point	Who decides	What they see	What happens next
The score	The reader	Three metrics, the task-set version, and the run letter	—

A.3 Residual risks accepted

Seven are enumerated in `SECURITY.md`. The three that most constrain use: the answer-support gap (a citation need not support its claim); six unmeasured injection payloads; and the unreviewed refusal wording.

Appendix B — AI-use declaration

Tools used. Claude (Anthropic), used conversationally throughout the project as a pair-programming and review partner.

What it was used for.

- Drafting and refactoring source code, including the dispatcher's gate structure, the serving layer, the scoring module and the test suite.
- Drafting this report and the repository's `README.md`, `FINDINGS.md` and `SECURITY.md`.
- Reviewing the author's reasoning, and — on several recorded occasions — contradicting it.

What it was not used for. No experimental result in this report was produced by it. Every evaluation run, traffic run and test run was executed by the author on the author's own machine, and the raw outputs are committed to the repository.

How the output was verified.

- **By running it.** Every command quoted in this report and in the README was executed, not assumed. This caught a real defect: the commands in section 7 were written as `.venv\Scripts\python.exe` and fail in PowerShell, which is the default shell on Windows, before reaching the interpreter. They were corrected only because the command was run rather than trusted.
- **By testing it.** 205 tests, written alongside the code, caught three defects that reading the output would not have: the truncating percentile, the claimed-action guard appending instead of replacing, and the step-watcher's ability to influence an outcome.
- **By checking the numbers against their source.** Two figures in an earlier draft of this report — a module count and a line count — were estimates. A verification script against the repository corrected both (16 → 18 modules, ~3,500 → 7,841 lines across the project).
- **By recording the errors rather than removing them.** `FINDINGS.md` lists twenty-four instruments in this project that reported something false, including one overstatement in the findings record itself, which was corrected the same day and left in the record.

Assessment. The assistance was substantial on code and prose. The judgement about what to measure, which results to trust, and which claims this report declines to make is the author's, and the findings record — which documents the author's own measurement code reporting falsely on ten separate occasions — is the evidence of that judgement rather than of its absence.